

CC — Cellular-Controlled Car

Overall Vision

The goal of CC is to design and implement a cellular-enabled RC car, modified to run off a Raspberry Pi, for remote driving from anywhere with cell coverage. Driving is done via a Meta Quest in HDMI passthrough mode with an Xbox controller, allowing remote control of the car in a fun and memorable way over long distances. The system combines robotics, software, and an immersive control scheme, and consists of four primary components: **the Hardware Control Container**, **the Video Capture and Streaming Container**, **the Master API**, and **the Frontend** (Quest display + controller input relay). Each component is modular, allowing independent development and testing while ensuring safe and synchronized operation.

Video Capture and Streaming Container Goal

The Video Capture and Streaming Container handles the camera feed and streams it directly to the user. It allows users to see the car's POV in real-time and provides video recording functionality for playback. The container supports monitoring of health and performance, ensuring smooth operation.

Key Features:

- Capture live video
- Stream video feed over the internet
- Error reporting
- Record video to persistent storage
- Monitor container and stream health
- API for communication

Hardware Control Container Goal

The Hardware Control Container is responsible for all physical interaction with the car. It **controls the steering servo and drive motor**, and ensures safe operation through physical limits and motion smoothing. This container includes calibration routines to reset the steering servo and allows adjustable presets for speed. Safety features, such as **emergency stop**, **maximum steering angle limits**, and **command request limits**, will be built into the container to prevent burning out the servo, motor, or other accidental damage.

Key Features:

- Servo and motor control
- Motion smoothing and calibration
- Safety mechanics
- Container health check
- API for communication
- Preset to reset servo / motor
- Different speed presets
- Error reporting

Master API Goal

The Master API acts as the central orchestrator of the system. It **manages commands from users, coordinates the hardware and video containers, and enforces system rules and state machines**. It ensures that only one user at a time can control the car while other users can observe via the spectator interface.

Key Features:

- Provide endpoints for servo commands, car control, video recording, and telemetry
- Enforce single-user control and session tokens
- Implement system state machine
- Log system events and collect telemetry for debugging
- System wide monitoring

Frontend Goal

The Frontend provides the user's interface for interaction: a Meta Quest in HDMI passthrough mode as the driver's display, paired with an Xbox controller relayed into drive commands, plus a spectator view for multiple users to view the live feed and telemetry. The frontend displays real-time system status, including location, steering angle, and recording status. Optional gamification features will make the interaction engaging for users.

Key Features:

- Interface for controlling the car
- Display system status, recording, and telemetry overlays
- Interactive and spectator view
- Optional gamified elements for fun and user engagement

Milestones

System Setup & Planning - (September & Early October)

- Pi setup, Wireguard setup, Git repo
- Research libraries and tools (OpenCV, FastAPI, etc)
- Document initial design and system architecture (wireframes and readmes)

Basic and Mock Setup - End of October

- Basic test setup built (wiring complete)
- Connect servo/motor and test basic movement commands
- Capture camera feed and stream a static frame or test video
- Skeleton API endpoints for servo, throttle, and video control
- Controller relay script connects to mock API; basic inputs (steering, throttle) registering

Core Functionality Complete - End of November

- Hardware: Servo and motor fully controllable
- Video: Live streaming operational, recording implemented
- Master API: State machine functional, single-user control enforced, endpoints tested
- Frontend: Real-time system status displayed, controller relay connected to API

Primary Goals Completed and Polishing - End of December

Hardware Container:

- Servo and motor mounted, calibrated, and fully controllable
- Motion smoothing and safety limits implemented
- Preset reset position and emergency stop functional
- Command requests limited

Video Container:

- Live streaming working
- Video recording implemented
- Stream health monitoring functional

Master API:

- State machine fully implemented
- Single-user control enforced, session tokens functional
- All endpoints fully completed
- Error and telemetry data collection implemented

Frontend:

- Real-time system status displayed (steering angle, speed, recording)
- All controls implemented and connected to Master API
- Gamification (speed modes, etc.)
- All functionality usable in application (movement, speed controls, etc.)
- Spectator and driver mode implemented

Final Integration, Polishing, Testing & Demo - Presentation Day

- Have a working product for Demo
- Final Documentation updates / cleanups

Tech Stack

Raspberry Pi with Python for servo and motor control, python for video capture and streaming, FastAPI for the Master API, a controller-relay script to bridge Xbox controller input into the Master API, and a Meta Quest in HDMI passthrough mode as the driver's display — all containerized with Docker and transmitted via WireGuard.

Current Team Members

Nick Pillsbury (Lead)

- pillsn@rpi.edu
- 4 credits

Mark Evans (Contributor)

- evansm2@rpi.edu
- 4 credits

Keith Wilfert (Contributor)

- wilfek@rpi.edu
- 4 credits